

# Set up **mabl** in your AI-DLC workflow

Go from zero to a working `mabl-verification` plugin inside an AI-DLC workflow on Kiro — then let it map your changes to mabl tests, run them, find coverage gaps, and gate the ship decision.

AI-DLC **2.6.105**

plugin **0.1.0**

harness **Kiro IDE**

model **Claude Opus 4.8**

## SETUP PATH

- What you're setting up
- Prerequisites
- 1** mabl CLI + auth
- 2** Connect the MCP server
- 3** Set your workspace
- 4** Base AI-DLC install
- 5** Install the plugin
- 6** Verify
  - Using it in a run
  - Worked example
  - Troubleshooting

# What you're setting up

Three layers compose at install time and separate cleanly: the workflow, the mabl methodology, and your project's context.

| LAYER                    | WHAT IT IS                                                                                                     | OWNER    |
|--------------------------|----------------------------------------------------------------------------------------------------------------|----------|
| AI-DLC                   | The staged idea→ship workflow ( <code>/aidlc</code> ) on the Kiro harness                                      | AWS Labs |
| mabl-verification plugin | <i>Methodology</i> — maps code changes to mabl tests, runs them, analyzes failures, finds gaps, gates the ship | mabl     |
| Your project             | <i>Context</i> — which workspace, which credentials, which app                                                 | You      |

The plugin adds a **6-question verification loop** to the Construction and Operation phases:

| Q  | QUESTION                                          | STAGE        |
|----|---------------------------------------------------|--------------|
| Q1 | Which tests are affected by this change?          | pre-pr       |
| Q2 | Do they pass?                                     | pre-pr       |
| Q3 | If not — why did it fail? (root-cause analysis)   | triage       |
| Q4 | What next? (fix / update test / retry / escalate) | triage       |
| Q5 | Is there a coverage gap?                          | coverage-gap |
| Q6 | Is it safe to ship?                               | ship-gate    |

## Prerequisites

Work through these before touching the plugin. Each has a one-line check.



### Node.js 22+

Check: `node -v`



### bun on your PATH

`bun --version` — install: `curl -fsSL https://bun.sh/install | bash`



### mabl CLI

`mabl --version` — install: `npm install -g @mablhq/mabl-cli`



### A mabl workspace with $\geq 1$ test covering your app

Ask your mabl SE if you don't have one



### Kiro IDE (v2.6+) with a base AI-DLC install

Verify: `/aidlc --doctor`



### Local dev server runnable

Tests execute against it — e.g. `npm run dev`

**! bun & non-interactive shells.** AI-DLC hooks run in shells that source `~/.zshenv` (zsh) or `~/.bashrc` (bash) — **not** `~/.zshrc`. If `which bun` works but the harness can't find bun, add bun's PATH export to `~/.zshenv`.

## STEP 1 Install & authenticate the mabl CLI

BASH

```
$ npm install -g @mablhq/mabl-cli
$ mabl --version
```

Authenticate. OAuth (browser) is simplest for a workshop:

BASH

```
$ mabl auth login
$ mabl auth info      # "Logged in as user [you@company.com]"
```

**i** Agent-to-agent / CI use an API key instead: `mabl auth activate-key <api-key>` .

## STEP 2 Connect the mabl cloud MCP server

The plugin talks to mabl through the cloud MCP server at `https://mcp.mabl.com/mcp` .

In Kiro, add it to `.kiro/settings/mcp.json` (this project) or

`~/.kiro/settings/mcp.json` (all projects):

`.KIRO/SETTINGS/MCP.JSON`

```
{
  "mcpServers": {
    "mabl": {
      "url": "https://mcp.mabl.com/mcp",
      "type": "http"
    }
  }
}
```

Reconnect the server in Kiro and complete the **browser consent flow** when prompted. If your agent doesn't prompt, ask it: *"Help me authenticate the mabl MCP."* Then confirm by asking *"What is my default workspace?"*

## API-key variant (no browser flow, scoped to one workspace)

```
.KIRO/SETTINGS/MCP.JSON
```

```
{
  "mcpServers": {
    "mabl": {
      "url": "https://mcp.mabl.com/mcp",
      "type": "http",
      "headers": { "x-api-key": "${MABL_API_KEY}" }
    }
  }
}
```

**i Client differences.** Cursor / Windsurf use `"type": "streamable-http"`. Claude Code can add it in one command: `claude mcp add --transport http mabl https://mcp.mabl.com/mcp`. Reference API keys from an env var — never commit them.

### STEP 3 Set your default workspace

Point the CLI at the workspace holding your tests. The config key is `workspace` — **not** `workspace-id`:

```
BASH
```

```
$ mabl config set workspace <your-workspace-id>
$ mabl config list      # confirm the "workspace" row
```

**! Gotcha.** The plugin's own README shows `mabl config set workspace-id` — that key name is wrong for the current CLI. Use `workspace`. (`--doctor` has the same cosmetic mismatch; it's advisory, not blocking.)

Find your workspace ID by asking the MCP agent *"What workspaces do I have access to?"*, or read it from the mabl app URL. Confirm the workspace has at least one test — the loop needs something to match and run.

---

#### STEP 4 Install the base AI-DLC framework

Skip if the workshop project already has a `.kiro/` + `aidlc/` install (check with `/aidlc --doctor`). Otherwise, for Kiro IDE:

BASH

```
$ git clone --branch main https://github.com/awslabs/aidlc-workflows.git
$ cd aidlc-workflows

$ mkdir -p your-project/.kiro your-project/aidlc
$ cp -R dist/kiro-ide/.kiro/. your-project/.kiro/
$ cp -R dist/kiro-ide/aidlc/. your-project/aidlc/
$ cp dist/kiro-ide/AGENTS.md your-project/AGENTS.md
```

Open `your-project` in Kiro IDE and select **Claude Opus 4.8** as the chat model (a paid Kiro plan is required for it).

---

#### STEP 5 Install the mabl-verification plugin

For Kiro, copy the packaged distribution into your project and run the composer once:

BASH

```
$ git clone https://github.com/andwaitforit/mabl-verification-plugin.git
$ cp -r mabl-verification-plugin/dist/plugins/mabl-verification/kiro-ide/.
<project>/

$ AIDLC_PLUGIN_ROOT="$PWD/mabl-verification-plugin/dist/plugins/mabl-
verification/kiro-ide" \
  AIDLC_PROJECT_DIR="<project>" \
  AIDLC_HARNESS_DIR=.kiro \
  bun <project>/hooks/compose.ts
```

**i** A **SessionStart** hook in the projection composes the plugin into your AI-DLC install automatically on the next session — no hand-editing of skill or prose files.

## STEP 6 Verify the install

Inside your Kiro AI-DLC session:

KIRO SESSION

```
> /aidlc --doctor
```

Expect a `Plugin check (mabl-verification)` line with **0 failures**. A healthy install shows:

✓ mabl CLI installed & authenticated

✓ Workspace configured

✓ 2 sensors composed

mabl-run-status, mabl-coverage-threshold

✓ The mabl-verification agent present

✓ The mabl-verification-validation scope present

✓ 3 plugin stages added on top of core

## Using it in a run

Start work as normal — the mabl stages fire on their own when an active scope includes them.

KIRO SESSION

```
> /aidlc Add a show/hide password toggle to the login screen
```

| STAGE                          | PHASE               | WHEN                                          |
|--------------------------------|---------------------|-----------------------------------------------|
| build-and-test (smoke)         | Construction        | Quick match-a-test-and-run after build        |
| mabl-verification-pre-pr       | Construction · 3.90 | After build-and-test, once mabl is configured |
| mabl-verification-coverage-gap | Construction · 3.95 | When pre-PR reports zero / partial coverage   |
| mabl-verification-ship-gate    | Operation · 4.50    | Final ship recommendation                     |



**i** **Scopes that activate the plugin:** `enterprise` , `feature` , `mvp` , `classic` , `mabl-verification-validation` . Runs scoped `poc` or `bugfix` need an explicit override — `/aidlc --scope feature ...` . The `mabl-run-status` sensor blocks on a failed run; `mabl-coverage-threshold` surfaces gaps without stopping the ship.

## What a run looks like

From the plugin's end-to-end validation — a password-visibility toggle on the login screen, run at `feature` scope.

| Q    | RESULT                                                                                                                                             |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Q1   | Semantic matching found the existing <i>"Mabl Bank / Auth: Admin can sign in"</i> test — the toggle wraps the same password field.                 |
| Q2   | <b>PASSED</b> ran locally in 21.85s, cloud-confirmed, no regressions.                                                                              |
| Q3-4 | N/A — nothing failed, no triage needed.                                                                                                            |
| Q5   | The toggle itself has no dedicated test. <b>NORMAL</b> severity, recommendation <code>author</code> — advisory under <code>standard</code> policy. |
| Q6   | See the gate below.                                                                                                                                |

◆ **SHIP**

**95**<sub>/100</sub>

Release readiness 95/100. Human actions left open: open the PR, and author a toggle test post-merge (non-blocking).

# Troubleshooting

| SYMPTOM                               | FIX                                                                                                                      |
|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Harness can't find <code>bun</code>   | Add bun's PATH export to <code>~/.zshenv</code> / <code>~/.bashrc</code> , not <code>~/.zshrc</code>                     |
| <code>--doctor</code> flags workspace | Set it with the <code>workspace</code> key: <code>mabl config set workspace &lt;id&gt;</code>                            |
| MCP tools not available               | Reconnect the server in Kiro, complete browser consent; check <code>mabl auth info</code> hasn't expired                 |
| Plugin stages don't fire              | Confirm scope is enterprise / feature / mvp / classic — <code>poc</code> / <code>bugfix</code> need <code>--scope</code> |
| No tests matched                      | Confirm the workspace has $\geq 1$ test covering the changed flow                                                        |
| Login expired mid-run                 | <code>mabl auth login</code> again — OAuth tokens are short-lived                                                        |

[plugin repo](#) ➤ [aidlc-workflows](#) ➤ [cloud MCP setup](#) ➤ [mabl CLI setup](#) ➤

**Other harnesses.** Claude Code & Codex install from the marketplace instead of copying a dist folder:

CLAUDE CODE / CODEX

```
> /plugin marketplace add andwaitforit/mabl-verification-plugin
> /plugin install aidlc-mabl-verification@mabl-plugins
```